

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1-CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1-Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A
Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I Gayathri .B.R (192521330) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: R.Asif

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts

Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided
---------------	-----	--	--	-------------------------------	---------------------------

1. Real-Time Embedded I/O Communication Mechanism

Given Constraints

- CPU utilization must be below 15%.
- Multiple sensors generate interrupts every 1 ms.
- Vectored interrupt handling must be used.
- Memory-mapped I/O and I/O-mapped I/O must be compared for low latency.

Problem Understanding

The system must collect data from multiple sensors at a very high frequency. Since interrupts occur every 1 ms, the CPU cannot spend too much time identifying and servicing interrupts. Therefore, a fast interrupt mechanism and efficient I/O access method are required.

Constraint Analysis

- **CPU utilization < 15%:** CPU overhead must be minimized.
- **Interrupt every 1 ms:** Interrupt handling must be very fast.
- **Vectored interrupts:** Each device should have a specific interrupt service routine (ISR).
- **Low latency:** The I/O addressing method should provide fast register access.

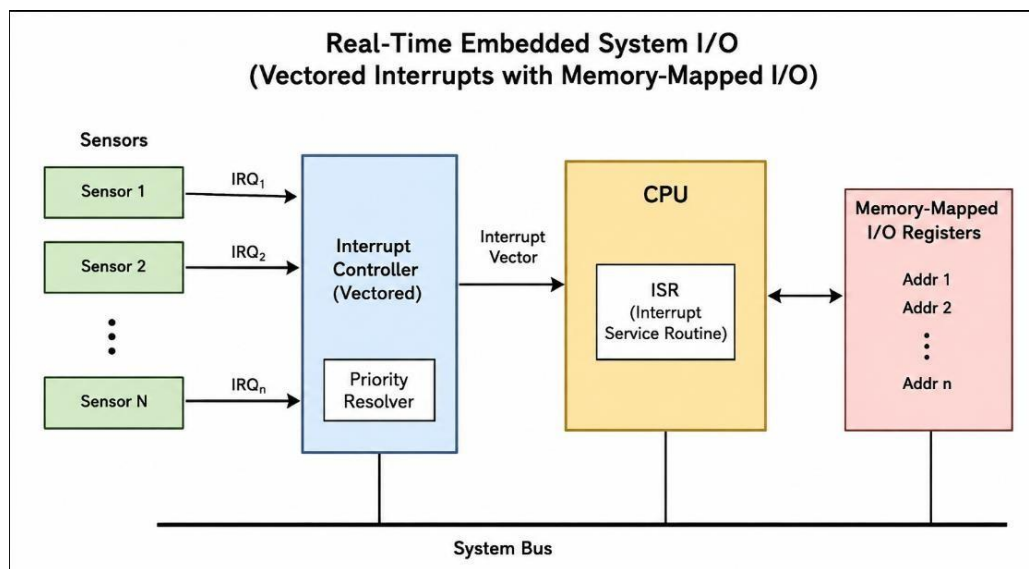
Solution Development

A vectored interrupt controller with memory-mapped I/O is selected.

Each sensor is connected to the interrupt controller. When a sensor generates an interrupt, the controller identifies the interrupt source and sends the corresponding vector to the CPU. The CPU directly executes the appropriate ISR.

The ISR performs only essential operations such as reading the sensor value, clearing the interrupt flag, and storing the data in memory. Complex processing is performed later to reduce CPU utilization.

Diagram:



Architecture

Sensors → Interrupt Controller → CPU → Memory-Mapped I/O Registers

Memory-Mapped vs I/O-Mapped I/O

Feature	Memory-Mapped I/O	I/O-Mapped I/O
Address space	Shared with memory	Separate I/O space
Instructions	Normal load/store	Special I/O instructions
Programming	Easier	More specialized

Register access	Fast and flexible	Architecture dependent
Selection	Preferred	Less preferred

Justification

Memory-mapped I/O is selected because sensor registers can be accessed using normal memory instructions. Vectored interrupts avoid software polling and directly transfer control to the required ISR. This reduces interrupt-processing overhead and helps maintain CPU utilization below 15%.

Conclusion

The proposed memory-mapped I/O + vectored interrupt controller provides fast sensor access, low interrupt latency, and reduced CPU overhead.

2. High-Speed NVMe I/O Subsystem

Given Constraints

- Storage throughput must be greater than 3 GB/s.
- NVMe over PCIe must be used.
- CPU should not perform bulk data transfers.
- DMA must be integrated with interrupt notification.

Problem Understanding

The system requires very high-speed storage communication. Moving every byte through the CPU would create significant CPU overhead. Therefore, a high-speed PCIe connection and DMA-based data transfer are required.

Constraint Analysis

- **Throughput > 3 GB/s:** Requires a high-bandwidth interface.
- **NVMe over PCIe:** NVMe provides efficient communication with highspeed SSDs.
- **Low CPU involvement:** Bulk data must bypass the CPU.
- **DMA + interrupt:** DMA performs transfer while interrupts notify the CPU after completion.

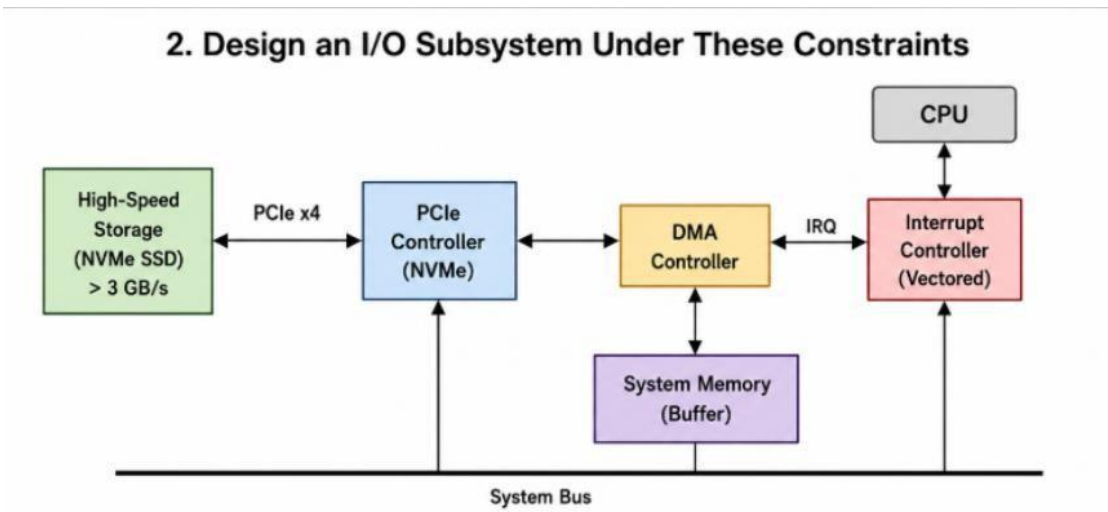
Solution Development

An NVMe SSD connected through PCIe with DMA is selected.

The CPU creates an NVMe command and places it in the command queue. The NVMe controller communicates through PCIe. DMA transfers data directly between the NVMe SSD and main memory.

After the transfer is completed, an interrupt is generated to notify the CPU.

Diagram:



Data Flow

CPU → NVMe Command Queue → PCIe → NVMe SSD

NVMe SSD → DMA → Main Memory

DMA Completion → Interrupt → CPU

Role of DMA

Without DMA:

NVMe → CPU → Memory

The CPU must participate in data movement.

With DMA:

NVMe → DMA → Memory

The CPU only controls the operation and handles completion notifications.

Justification

DMA is selected because the CPU does not need to copy large amounts of storage data. PCIe provides the required high-speed communication path, while NVMe provides efficient command queues for SSD operations.

Conclusion

The best solution is NVMe + PCIe + DMA + interrupt notification, which provides high throughput while keeping CPU involvement low.

3. Multi-Device USB and Thunderbolt Communication Architecture

Given Constraints

- Six peripherals must use USB.
- Two high-speed devices must use Thunderbolt.
- Interrupt starvation must be avoided.
- Hardware and software interrupts must be supported. □ Fair bus arbitration must be ensured.

Problem Understanding

The system contains eight devices using two different communication technologies. Thunderbolt devices require high-speed communication, while USB devices also need regular access. A suitable priority and arbitration mechanism is required so that no device waits indefinitely.

Constraint Analysis

- **6 USB devices:** Connected through a USB host controller.
- **2 Thunderbolt devices:** Connected through a Thunderbolt controller.
- **Interrupt starvation:** Low-priority devices must also receive service.
- **Hardware interrupts:** Used for actual device events.
- **Software interrupts:** Used for deferred/background operations. □ **Fair arbitration:** Bus access must be shared among devices.

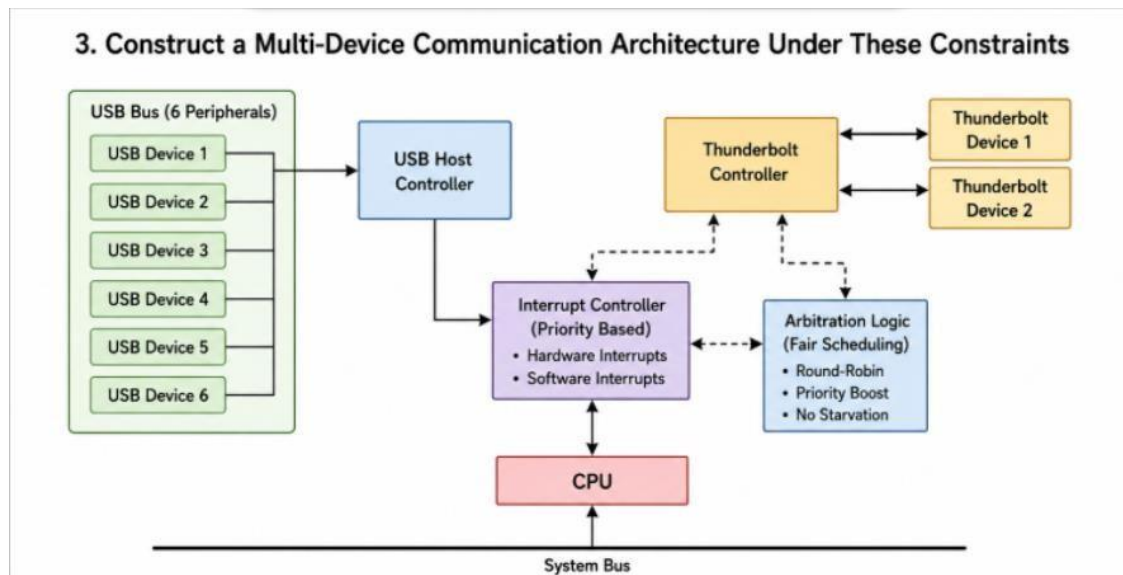
Solution Development

The six USB peripherals are connected to a USB host controller.

The two high-speed devices are connected to a Thunderbolt controller.

A priority-based arbitration mechanism is used. Thunderbolt devices can receive higher priority when required, but USB devices are given guaranteed service using round-robin scheduling and aging.

Diagram:



Arbitration

USB1 → USB2 → USB3 → USB4 → USB5 → USB6 → Repeat

If a USB device waits for too long, its priority is increased so that it cannot be permanently blocked.

Interrupt Handling

- **Hardware interrupts:** Device events such as data arrival, errors, or connection changes.
- **Software interrupts:** Background processing, deferred tasks, and operating system services.
- **Priority mechanism:** Critical events are handled first.
- **Round-robin:** Provides fair access among devices of similar priority.

Justification

Using only fixed priority could cause USB devices to suffer starvation. Using only round-robin could delay important Thunderbolt operations. Therefore, a **priority** + fairness mechanism provides both fast response and fair bus access.

Conclusion

The proposed system uses USB + Thunderbolt + hardware/software interrupts + priority scheduling + round-robin arbitration to provide reliable and fair communication.

4. Data Acquisition System – 500 MB/s

Given Constraints

- Continuous data stream of 500 MB/s.
- Minimal CPU intervention.
- DMA transfer must be compared with interrupt-driven I/O. □ Device registers must use memory-mapped I/O.

Problem Understanding

The data acquisition device continuously produces data at 500 MB/s. If the CPU handles every data item, CPU utilization will become very high. Therefore, an autonomous data-transfer mechanism is required.

Constraint Analysis

- **500 MB/s:** Requires high-speed continuous data transfer.
- **Minimal CPU intervention:** CPU should not copy data.
- **DMA comparison:** DMA should be evaluated against interrupt-driven I/O.
- **Memory-mapped I/O:** Device registers are accessed through memory addresses.

Solution Development

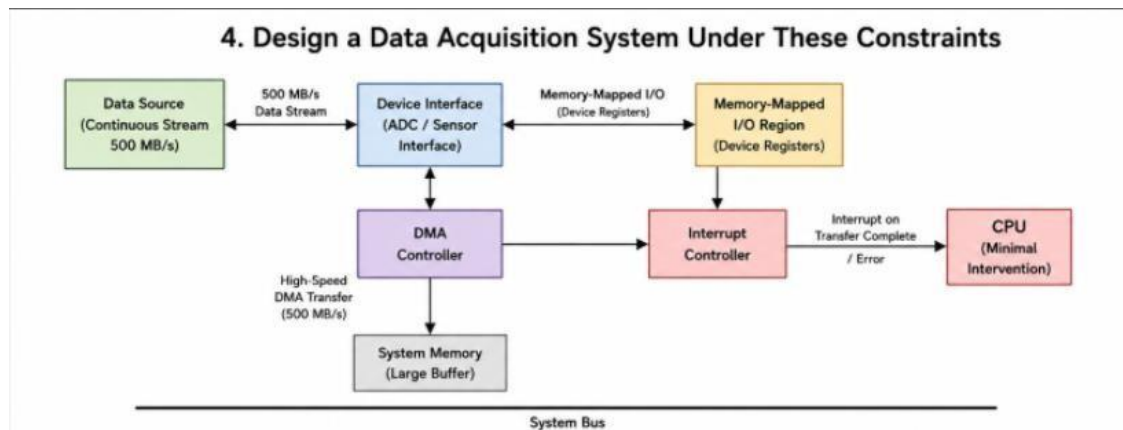
The recommended solution is memory-mapped I/O + DMA + double buffering.

The CPU configures the DMA controller with:

- Source address
- Destination address
- Transfer size

DMA then transfers data directly from the acquisition device to memory.

Diagram:



Data Flow

Data Acquisition Device → DMA Controller → Buffer A / Buffer B → CPU

While DMA fills Buffer A, the CPU processes Buffer B. After Buffer A becomes full, the roles are exchanged.

DMA vs Interrupt-Driven I/O

Parameter	DMA	Interrupt-Driven I/O
CPU involvement	Very low	High
Data movement	Device → Memory	Device → CPU → Memory
Suitable for 500 MB/s	Yes	No
Interrupt frequency	Block-based	Very high
CPU overhead	Low	High
Performance	High	Lower

Justification

At 500 MB/s, generating an interrupt for every small data unit would create an extremely large number of interrupts. DMA can transfer large blocks and generate only a completion interrupt, greatly reducing CPU overhead.

Conclusion

Memory-mapped I/O + DMA + double buffering is the most suitable solution for continuous 500 MB/s data acquisition.

5. Nested Interrupt Handling Mechanism

Given Constraints

- Nested interrupts must be supported.
- Maximum interrupt latency must be less than 10 μ s.
- Vectored interrupts must be used for high-priority devices. □ Software interrupts must be used for background tasks.

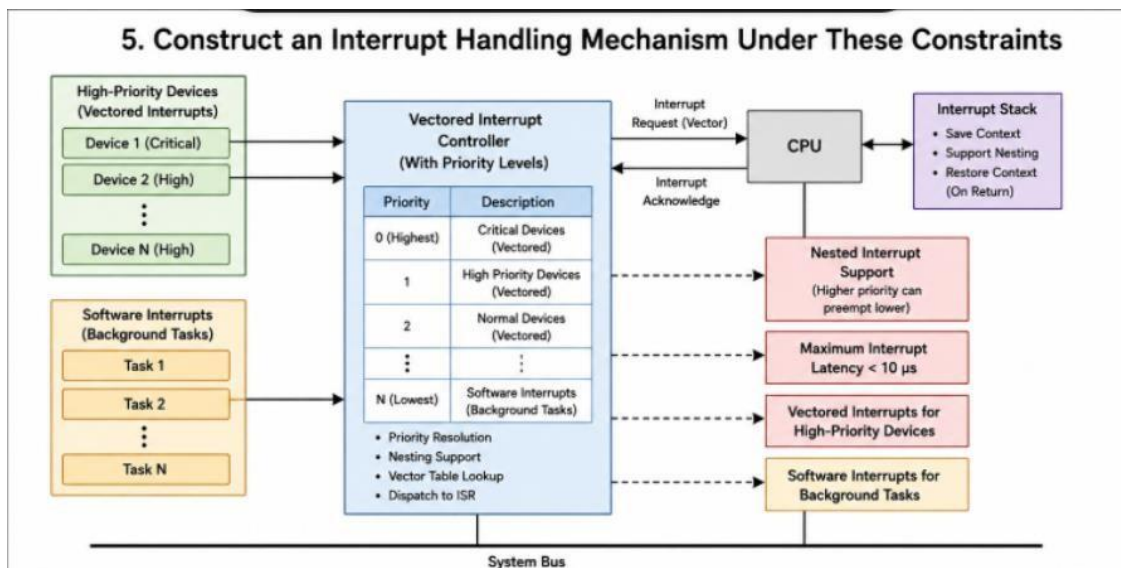
Problem Understanding

A high-priority interrupt may occur while the CPU is already processing a lower-priority interrupt. The system must temporarily suspend the lower-priority ISR, service the important interrupt, and then return to the previous ISR.

Constraint Analysis

- **Latency < 10 μ s:** Interrupt dispatch must be very fast.
- **Nested interrupts:** Higher-priority interrupts must preempt lower-priority ISRs.
- **Vectored interrupts:** Directly identify the required ISR.
- **Software interrupts:** Used for non-critical background operations.

Diagram:



Solution Development

A priority-based nested interrupt controller is used.

When an interrupt occurs, the controller checks its priority. A high-priority interrupt is immediately directed to its corresponding ISR using a vector.

If a higher-priority interrupt arrives while another ISR is executing:

Low Priority ISR → High Priority Interrupt → High Priority ISR → Return → Low Priority ISR

The CPU saves the current ISR context before executing the higher-priority ISR.

Priority Structure

Priority	Task
1	Emergency / Real-Time Device
2	High-Speed I/O
3	Normal I/O
4	Background Tasks

Reducing Interrupt Latency To

maintain latency below 10 μ s:

- Use vectored interrupts.
- Keep ISRs short.
- Use priority-based preemption.
- Minimize interrupt masking.
- Save and restore CPU context efficiently.
- Move lengthy processing to software/deferred interrupts. □ Use DMA for large data transfers.

Software Interrupts

Software interrupts are used for:

- Background processing
- Logging
- Deferred cleanup
- Task signaling

They are given lower priority so they do not interfere with critical hardware interrupts.

Conclusion

A nested, priority-based vectored interrupt system provides fast response to critical devices while software interrupts handle background activities. This design helps maintain the required less than 10 μ s interrupt latency.